

Day-16 | Quantum Machine Learning

Training Models with Quantum Circuits

1. Learning Objectives

- ✓ Understand how quantum computing and machine learning complement each other.
- ✓ Explain the workflow of a Quantum Machine Learning (QML) model.
- ✓ Differentiate between classical and quantum machine learning.
- ✓ Learn the importance of quantum data encoding (Feature Maps).
- ✓ Recognize the role of parameterized quantum circuits and classical optimizers.

2. Quick Recap

- **Optimization:** Finding the best possible solution to a complex problem.
- **Parameterized Circuits:** Quantum circuits with adjustable gate parameters (θ).
- **Hybrid Loop:** Classical computers update parameters to improve quantum circuit outputs.

Transition: Today, we use this exact hybrid optimization loop not to find the shortest path, but to **train machine learning models** to recognize patterns in data.

3. Why Do We Need Quantum Machine Learning?

Motivation: As classical ML models and datasets grow massive, quantum computers aim to exploit superposition and entanglement to discover hidden, highly complex data patterns more efficiently.

Real-World Applications: 2

- **Drug Discovery:** Molecular screening.
- **Finance:** Fraud detection & portfolio risk.
- **Cybersecurity:** Network anomaly detection.
- **Healthcare:** Medical image diagnosis (MRI).
- **NLP:** Complex text classification.
- **Science:** Climate & particle physics.

4. Revisiting the Basics

The Parameterized Quantum State

$$\psi(\theta)$$

Where:

- ψ represents the encoded quantum state.
- θ represents the **trainable parameters** (the "weights" of our quantum model).

5. Concept Expansion: Myths vs. Reality

Common Misconception

"QML will replace all classical ML."

Reality / Correction

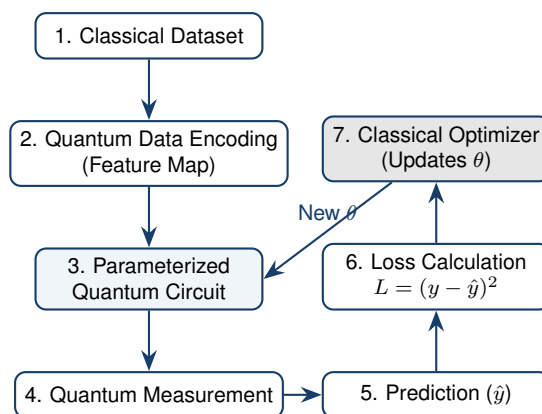
Correction: QML complements classical ML. It is used for specific, highly complex calculations, while classical GPUs handle everyday ML tasks.

"Training happens entirely on the QPU."

Correction: QML is a **hybrid** process. The QPU evaluates the data, and the classical CPU updates the parameters.

6. Core Mechanism (Intuition First): The QML Workflow

QML relies on transforming classical data into quantum states, rotating them, and measuring the outcome.



7. Computational Core: Classical vs. Quantum Elements

Component	Classical ML	Quantum ML
Data Unit	Bits (0 or 1)	Qubits (Superposition of 0 & 1)
Processing	Neural Networks (Matrix Math)	Quantum Circuits (Unitary Gates)
Input Format	Numerical Features (Tensors)	Quantum State Representations
Hardware	CPU's, GPU's, TPU's	QPU's (Superconducting, etc.)

8. Core Equations & Rules

To train a model, we must measure its error and update its parameters.

1. The Prediction (Expectation Value): $\langle O \rangle = \psi(\theta) O \psi(\theta)$

The predicted output generated by measuring the observable O on the final quantum state.

2. The Loss Function (Mean Squared Error): $L = (y - \hat{y})^2$

Where y is the actual true label, and $\hat{y}$ is our predicted expectation value. The optimizer's goal is: $\theta^ = \arg \min_{\theta} L(\theta)$*

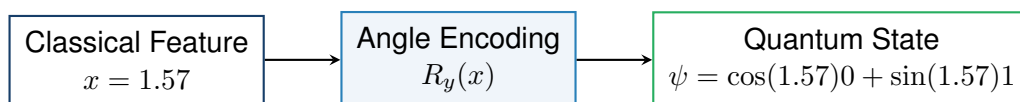
9. Growth & Convergence (How Learning Happens)

As the optimization loop repeats, the loss decreases, and accuracy grows.

Iteration (Epoch)	Circuit Parameters (θ)	Loss Value (L)
1 (Random Guesses)	$[0.15, -0.42, \dots]$	0.85 (High Error)
10 (Adjusting)	$[0.33, -0.21, \dots]$	0.42 (Improving)
50 (Converging)	$[0.78, 0.11, \dots]$	0.05 (Trained Model)

10. Visualizing Quantum Data Encoding

Classical data **cannot** directly enter a QPU. It must be encoded (mapped) into a quantum state.



11. Practical Implementation: VQC in Qiskit

Implementing a Variational Quantum Classifier (VQC) in Python:

```

from qiskit.circuit.library import ZZFeatureMap, RealAmplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit_algorithms.optimizers import COBYLA
from qiskit.primitives import StatevectorSampler

# 1. Encode Data & Build Trainable Circuit
feature_map = ZZFeatureMap(feature_dimension=2) # Encodes 2D classical data
ansatz = RealAmplitudes(2, reps=2)             # Parameterized quantum circuit

# 2. Setup Optimizer & Execution Engine
optimizer = COBYLA(maxiter=100)
sampler = StatevectorSampler()

# 3. Create & Train the Quantum Model
vqc = VQC(sampler=sampler, feature_map=feature_map,
           ansatz=ansatz, optimizer=optimizer)
vqc.fit(training_features, training_labels) # The Hybrid Training Loop!
  
```

```
# 4. Predict
predictions = vqc.predict(test_features)
```

12. Measurement & Outputs

Because quantum mechanics is probabilistic, the output is an **expectation value** over multiple shots. E.g., if predicting Species A or B, the circuit might return 0 85% of the time (Species A) and 1 15% of the time (Species B).

13. Worked Example: Flower Classification

Input: Petal length = 4.6 cm, Width = 1.5 cm (True Label: Species A).

- **Process (Epoch 1):** Encoded → Random parameters → Predicts Species B. Loss is HIGH. Optimizer adjusts θ .
- **Process (Epoch 50):** Encoded → Tuned parameters → Predicts Species A. Loss is LOW.

Output: The trained model accurately predicts unseen flower data.

14. ★ Key Takeaways

- > QML is a **hybrid workflow**: Quantum computers process data, classical computers optimize parameters.
- > Classical data must be explicitly mapped into a quantum state via **Data Encoding/Feature Maps**.
- > **Parameterized Quantum Circuits (Ansatz)** act as the trainable "neural network" layers in QML.
- > The model learns by iteratively minimizing a **Loss Function** (prediction error).
- > QML does not replace classical AI; it explores quantum properties to find patterns classical machines miss.